

Cooperative Reinforcement Learning for Energy Management in Multi-Hop Networks With Energy Harvesting

Hrishikesh Dutta[✉], *Member, IEEE*, Amit Kumar Bhuyan[✉], *Graduate Student Member, IEEE*,
and Subir Biswas[✉], *Senior Member, IEEE*

Abstract—This paper proposes a Reinforcement Learning (RL) framework for joint transmit-sleep scheduling for multi-hops wireless sensor and IoT networks with energy harvesting. The goal of the framework is to learn situation-specific scheduling policies for reducing energy expenditure, while maintaining network performance in terms of packet delivery ratio and end-to-end delay. The proposed system uses a cooperative RL approach where two learning agents, deployed per node, jointly learn transmit and sleep scheduling strategies to manage network energy budgets. The RL framework also uses a localized learning confidence parameter sharing strategy that allows the nodes to ignore unreliable RL observations. This makes the system scalable for network topologies where the source and destination nodes are separated by many numbers of hops. The learning module is decentralized in that learning is independently carried out at the sensor/IoT nodes without relying on a central learning coordinator. Decentralized learning makes the system computationally efficient, and also avoids energy and communication bandwidth overhead usually imposed by a central coordinator. With simulation studies, the proposed learning-driven protocol is tested and compared against existing known MAC sleep schedulers. The proposed mechanism is validated for different scenarios with heterogeneous topologies, traffic, and energy harvesting conditions.

Index Terms—Energy-harvesting, sleep scheduling, transmission scheduling, reinforcement learning, cooperative learning.

I. INTRODUCTION

ENERGY-CONSTRAINED wireless networks, particularly those utilizing energy harvesting, require efficient transmit-sleep scheduling to balance performance and network longevity. These networks face two critical challenges: reducing service disruptions caused by energy shortages and maintaining reliable performance in terms of throughput and delay. Traditional pre-programmed scheduling approaches often fail to adapt to dynamic conditions such as varying network traffic patterns, heterogeneous node requirements, and spatiotemporal variations in energy availability [1]. This inflexibility frequently leads to suboptimal resource utilization,

leading to wasted energy, and compromised balance between network performance and lifetime. To address these limitations, this paper proposes a Reinforcement Learning (RL) architecture for transmit-sleep scheduling in energy-harvested wireless networks with limited energy budgets. The proposed approach enables nodes to learn adaptive joint scheduling policies that respond to spatiotemporal energy variations and traffic dynamics, thus overcoming the constraints of traditional methods. This paradigm ensures an improved balance between network performance and energy efficiency, enhancing overall system reliability and sustainability.

There are existing reported works [2], [3], [4], [5] that deal with sleep scheduling in networks with energy constraints. The authors in [2] use a game-theoretic approach to find sleep-scheduling policy for solar powered sensor networks. In addition to the fact that these policies are static with respect to network traffic and topologies, they also do not consider transmission scheduling decisions. As will be shown later in this paper, transmission scheduling plays an important role in maintaining network performance in energy-harvested networks. There are RL-based approaches [3], [9] applied for sleep scheduling in energy harvested networks. Besides the above limitations arising from not considering policies for transmission strategies, these often rely on a centralized arbitrator for learning scheduling policies. Centralized learning, apart from being computationally inefficient and creating a burden on a central server, comes with additional bandwidth and energy costs for downloading learnt scheduling policies from the server to network nodes. Moreover, performance of the learnt policies heavily depends on the reliability of information collected from the sensor/IoT nodes over a possibly error-prone channel. Another limitation of the learning-driven scheduling approaches [9], [10] is that these are not suitable for networks with multiple hops. This is because of the fact that learning for multi-hops networks is challenging, since learning errors in a node can be carried over to all the downstream nodes in the topology it is part of.

In this paper, a decentralized and cooperative Reinforcement Learning [18], [19] approach for joint sleep-transmission scheduling is presented to overcome the issues mentioned above. The scheduling problem is first modeled as a Markov Decision Process (MDP) [20] and solved using Reinforcement Learning as a temporal difference solution. The objective

Received 16 February 2024; revised 6 December 2024 and 22 January 2025; accepted 14 February 2025. Date of publication 20 February 2025; date of current version 27 November 2025. The editor coordinating the review of this article was K. Wang. (Corresponding author: Hrishikesh Dutta.)

The authors are with the Electrical and Computer Engineering Department, Michigan State University, East Lansing, MI 48823 USA (e-mail: duttahr1@msu.edu; bhuyanam@msu.edu; sbiswas@egr.msu.edu).

Digital Object Identifier 10.1109/TGCN.2025.3544073

is to efficiently manage the ultra-thin energy budget of energy-harvesting sensor/IoT nodes, while maintaining acceptable network performance. This is realized by cooperative learning behavior achieved by two RL agents deployed per node. Each of these agents jointly learn scheduling policies for the node in order to achieve the above-mentioned objective. This learning architecture is supported by the online sharing of a learning confidence parameter by each node. Sharing of this confidence parameter assists decentralized learning of the scheduling policy for the downstream nodes in the network topology. In this way, each IoT/sensor node learns a transmit-sleep scheduling policy independently in a decentralized manner. This cooperative learning architecture is suitable for learning in networks with multiple hops between source and destination, where learning performance for a node is heavily dependent on the scheduling policies learnt by other nodes in the topology, which makes the proposed system scalable with network size.

The paper has the following specific scopes and contributions. First, a cooperative multi-agent Reinforcement Learning architecture is proposed for learning joint transmit-sleep scheduling decisions in energy-harvested wireless networks. The developed framework is decentralized in the sense that each node learns its scheduling policy independently, without the requirement of a central arbitrator. Second, an online learning confidence sharing model is proposed for making the learning architecture scalable for large networks with multiple hops. Third, with extensive simulations, a detailed characterization of the proposed architecture is done for different traffic patterns, and it is compared against known benchmark scheduling policies.

II. RELATED WORK

Efficient sleep-scheduling for energy management in wireless networks has always been a point of concern for researchers. The authors in [2] propose transceiver on/off scheduling policies based on available energy budget, queue status and channel characteristics of the solar energy harvested wireless sensor networks. This work does not take transmission-scheduling for energy management into consideration, which would require adjustment of policy parameters based on traffic scenarios. Reinforcement Learning has been used in the literature for energy management in wireless networks. The works in [4], [5], [6], [7] develop learning-driven approaches for sleep scheduling in sensor networks. These mechanisms mainly deal with finding a suitable sleep schedule for sensor nodes using centralized learning techniques. One common limitation of these works is that they rely on a centralized server for learning the policies for the entire network. Apart from the fact that centralized learning puts computation burden on the central controller, these techniques also require additional energy and bandwidth for communicating the policies, and for learning observables to and from the server. Similar limitations of centralized computation exist in the RL-based energy management solutions in [8], [9] for rechargeable and energy-harvested networks.

There are papers [14], [15], [16], [17] that deal with energy management in sensor networks by adaptively selecting nodes for sleep scheduling. By implementing a dynamic restructuring of clusters of sensor nodes and a fuzzy-based sleep scheduling approach, the paper [14] aims at reducing the energy consumption of the network. Similar fuzzy logic-based clustering of sensor nodes for energy management is proposed in [15] to increase the lifespan of the sensor network. An RL-driven sleep scheduling mechanism for cooperative computing method is developed in the paper [16] for energy efficiency improvement. The paper develops a computing node selection algorithm for enhanced battery lifetime. A similar energy management scheme using deep RL for sleep scheduling and computation and communication resource allocation is proposed in [17]. The developed architecture is shown to be applicable for 5G Mobile Edge Computing Networks for Industrial IoT. One drawback of these adaptive node selection-based energy management approaches is that for heterogeneous traffic distribution, certain sensor nodes with high data rate may not be able to get channel access, leading to an increase in delay. The work in [4] proposes an RL algorithm for controlling the access time and transmit power of the sensor nodes in WSN. The paper shows that the proposed mechanism improves Quality-of-Service, achieves high energy efficiency and transmission reliability. The work in [10] has developed a resource-scheduling mechanism using node-level deep Reinforcement learning in order to improve transmission reliability. The authors in [11] propose an RL-based solution that helps resource-constrained nodes enhance their performance by saving battery power and maintaining the quality of transmitted data. As pointed out in the previous scheduling-based methods, the authors here do not consider optimizing the transmission scheduling decisions, which play a significant role in efficient energy management. Moreover, the works in [3], [4], [9], [10], [12] are mainly designed for networks with star topologies and with limited number of hops between the source and the destination of a traffic flow. This puts limitations on the applicability of these mechanisms for practical networks with multiple hops. A summary of different related works is discussed in Table I.

In this paper, a decentralized and cooperative multi-agent RL-driven solution is proposed for energy management in energy-harvested wireless networks with ultra-thin energy budgets, while considering the limitations detailed above. Nodes learn scheduling policies independently, without a central controller, while cooperative learning ensures an optimal balance between energy efficiency and network performance in multi-hop flows. The two-agent system jointly optimizes transmit-sleep scheduling for reliable communication, and the distributed approach enhances scalability in large networks.

Cooperative RL has been explored in literature in a range of applications involving multi-agent systems. In [26], the authors have developed an RL-driven cooperative service caching and task offloading mechanism for Digital Twin Edge Networks using a graph attention-based approach. Multi-agent Reinforcement Learning for finding optimal energy management strategy in hybrid electric vehicles in a cooperative manner is proposed in [27]. A group-based cooperative RL

TABLE I
RELATED WORKS REVIEW

Paper	Objective	Methodology used	Network Model	Implementation
[2]	Packet drop reduction	Game-theoretic approach	Single node	Decentralized
[4-6], [10]	QoS improvement, energy management	Machine Learning	Star topologies, Single/multi-hops	Centralized
[8, 9]	Energy Efficiency improvement	Reinforcement Learning	Star topologies	Centralized
[11, 12]	Resource allocation, energy management	Transmission strategy management using RL	One-hop star topologies	Decentralized
[14, 15]	Network lifetime improvement	Fuzzy-based scheduling and transmission control	Star network clusters	Centralized
[13]	QoS improvement	QoS-aware slot allocation, power-rate control	One-hop star topologies	Centralized
[16, 17]	Energy efficiency improvement	RL-driven scheduling, resource allocation	Star network clusters	Centralized

solution for last mile logistics is developed in [28]. Similarly, the authors in [29] have proposed a multi-UAV cooperative search algorithm using RL. Cooperation is achieved through different means, such as, by sharing a common reward function, value function factorization, having a common state observable space for the RL critics, [30] etc. The common idea, however, in all these approaches is having a common optimization objective for all the agents.

III. SYSTEM MODEL

A. Network and Traffic Model

To maintain generality, we consider a multi-point-to-point and multi-hop network model in which the wireless sensor nodes/IoT devices send data to a base station using multi-hop routes. At the MAC layer, it is assumed that the network time is synchronized, slotted, and the MAC frames are of fixed size, which is dimensioned *a priori* based in the degree of the network topology. Application layer packet generation at the source nodes follow Poisson distribution with packet generation rate λ packet per frame (PPF). Each node maintains an $M/G/1/K$ buffer/queue, where the Poisson distributed queue arrival rate is governed by λ , and the queue service rate is determined by the transmission-sleep policy actuated by the proposed learning mechanisms as presented in Section IV.

B. Energy Harvesting and Consumption Model

In the proposed framework, the wireless sensor/IoT nodes rely on energy harvesting for powering their communication subsystem. While the proposed mechanism would scale for any source of energy harvesting, in this paper, we have considered a solar energy harvesting model to demonstrate the efficacy of the proposed mechanisms. A 2-state Markov Model is used for simulating solar energy harvesting [2]. The states of the Markov model are (1) low radiation state, in which sunlight is blocked by clouds and hence, radiation is not enough to charge the node batteries, and (2) high radiation state, in which there is direct sunlight and is sufficient to charge the battery. This is represented by the transition probability matrix

R , where state 1, 2 represent high and low radiation states respectively:

$$R = \begin{bmatrix} R_{1,1} & R_{1,2} \\ R_{2,1} & R_{2,2} \end{bmatrix} \quad (1)$$

Assuming that the cloud size is exponentially distributed with mean ' C ' m and the wind speed is w_s m/s, the elements of matrix R can be obtained using the analytical model in [2]:

$$\begin{aligned} R_{1,2} &\approx \left(\frac{1}{\mu_s}\right)t \times e^{\left(\frac{1}{\mu_s}\right)t}, \\ R_{2,1} &\approx \left(\frac{1}{\mu_c}\right)t \times e^{\left(\frac{1}{\mu_c}\right)t}, \\ R_{1,1} &= 1 - R_{1,2} \text{ and } R_{2,2} = 1 - R_{2,1} \end{aligned}$$

Here, $\mu_c = \frac{C}{w_s}$, $\mu_s = \frac{C \times (1 - P_c)}{w_s(P_c)}$ are the average time for which the radiation is low and high respectively; P_c is the probability of solar radiation in low state and t is the length of a time frame. Each node has a battery capacity of B units, where a packet transmission consumes one unit of battery. Thus, the battery status at time t is given by $b \in \{0, 1, 2, \dots, B\}$, where the battery is charged in high radiation state with probability P_{charge} . When $b = 0$, battery is completely depleted and needs recharging. If $b = B$, the battery is fully charged and, the recharging circuitry is turned off.

The energy consumption model in [13] is considered for radio hardware energy dissipation. In this model, power consumed while transmitting and receiving a packet are given by Eqns. (2) and (3) as follows:

$$P_T = (1 + \alpha_T) \times E_{tx} + E_{ct} \quad (2)$$

$$P_R = E_{ct} \quad (3)$$

Here, E_{tx} represents the transmission power of the transmitter with amplifier inefficiency factor α_T , and E_{ct} is the circuitry power consumption, which is a constant depending on a specific transmitter. In this paper, we assume constant power for both transmission (E_{Tx}) and reception (E_{ct}), the values of which can be decided based on the physical hop distance between the nodes. In addition to the communication energy

consumption modeled in Eqns. (2) and (3), there is additional energy expenditure (P_{comp}) involved in executing the learning model used for transmit-sleep scheduling. In this work, we use the baseline Q-learning for executing the scheduling framework, owing to its low computation complexity. The Q-table update rule is implemented using the standard Bellman-Ford equation [24], which basically involves two multiplications, three additions and $|\mathcal{A}| - 1$ comparison operators, where $|\mathcal{A}|$ is the size of the action space. As demonstrated in [24], the energy required for executing such operations is in the order of $10^{-12}J$, whereas the communication energy for transmitting and receiving a single packet is in the order of $10^{-3}J$ [13], [25]. That is, $P_{comp} \ll P_T, P_R$ and can be ignored while computing energy consumption.

IV. COOPERATIVE REINFORCEMENT LEARNING FOR JOINT TRANSMIT-SLEEP SLEEP SCHEDULING

Fig. 1 shows a high-level working model of the RL-based architecture for a node ' n ' in an energy-harvesting wireless network. There are two RL-agents deployed in each node: a *Transmission Scheduling Agent* and a *Sleep Scheduling Agent*. For both, the RL environment is the network itself. Both the agents share the same reward function which is computed from the RL-related observable variables including harvested energy and network performance parameters. The state definition for those agents, however, are different and independent. As indicated by the dotted arrows, the state for the transmission scheduling agent is perceived directly from the network observables, whereas the state for the sleep scheduling agent is dependent on the RL-policies of the transmission scheduler. Using these independent notions of states, these two agents cooperatively learn the node's transmit and sleep schedules.

The use of two learning agents per node arises from the need to address scheduling decisions at two hierarchical levels. The primary task involves determining an efficient sleep schedule, specifying when a node's transceiver should power on or remain off to conserve energy. At the secondary level, the node must decide its actions during active periods, including scheduling transmissions of queued packets and listening for neighbor transmissions. This is to make sure that energy expenditure is reduced, while maintaining a stable MAC packet queue (i.e., low latency), and reducing missed packet receptions. This two-layer scheduling problem is modeled as a multi-objective optimization, addressed through the collaborative operation of the two RL agents per node. The agents cooperatively find a transmit-sleep schedule by sharing the common objectives (detailed above). This cooperation is achieved by maximizing the long-term expected reward governed by the same reward function.

An important feature of the proposed learning architecture is the sharing of an online learning confidence parameter with the one-hop neighbors of each node. In the figure, this is depicted by the *Learning confidence Parameter (LCP) sharing*, indicated by the red arrows. This parameter, which is piggybacked in the MAC PDU, is used to prevent unreliable

learning parameter update by the RL agents. Another parameter piggybacked in the PDU is the *Hop Index (HI)*, which is defined as the number of hops a node is away from the source of a passing flow. The figure shows examples of two incoming and outgoing MAC PDUs for node n . One received from its one-hop neighbor ' $n - 1$ ', and the other destined for node ' $n + 1$ '. These two characteristics of the proposed framework, namely, 1) cooperatively learning the joint transmit and sleep schedules by the two RL agents, and 2) inter-node sharing of the confidence parameter to assist learning by the RL agents, give rise to cooperative learning behavior of the proposed approach. The detailed working of the two RL agents and their cooperative nature are explained below.

A. Transmission Scheduling Agent

The function of this agent is to learn a suitable transmission schedule for its host node. Finding a suitable transmission schedule can be tricky in that a conservative transmission periodicity can lead to excessive queue/buffer growth even though it will be favorable for better energy conservation. Having an aggressive schedule, on the other hand, can lead to low packet delay and drops (i.e., smaller queues) at the expense of excessive energy consumption, which may lead to a shutdown of the node due to exhausted energy that was harvested. Therefore, the objective is to develop a transmission schedule that can balance those two effects and be adaptive with the spatiotemporal characteristics of energy harvesting.

The MDP action space ($\mathcal{A}_T$) for this agent is defined by the probability of transmitting a packet (P_{tx}) in the queue. To keep the action space discrete, this probability is quantized into $|\mathcal{A}_T|$ discrete values in the range $[0, 1]$, where $|\mathcal{A}_T|$ denotes the cardinality of the action space. These actions are selected using ϵ -greedy policy in an RL decision epoch set to a duration of h frames. The state space ($\mathcal{S}_T$) as perceived by the agent is represented by the energy influx to the node resulting from harvesting. Similar to the packet transmission probability, the harvested energy is also discretized into $|\mathcal{S}_T|$ distinct ranges. Formally, the state as perceived by an agent for node n at an epoch t is defined as $s_{n,Tx}(t) = g(\frac{1}{h} \sum_{k=t-h}^t E_{in,n}(k))$. Here $E_{in,n}(k)$ is the energy harvested in frame k by node n and $g(x)$ is the function for quantization as defined by:

$$g(x) = \begin{cases} \mathcal{I}_T, & \text{if } -0.5 \leq \mathcal{I}_T - 0.5 \leq 10x < \mathcal{I}_T + 0.5 \leq 8.5 \\ 9, & \text{if } x \geq 0.85 \end{cases} \quad (4)$$

Here $\mathcal{I}_T \in I$ and $\mathcal{I}_T \geq 0$. In this work, tabular Q-learning [20], [21] is used for solving the MDP, owing to the limited computation, storage, and processing capacity of the resource-constrained IoT/sensor nodes.

The reward for the agent in node n at an epoch t in this setting is given by Eqn. (5).

$$\mathcal{R}^{(n)}(t) = \begin{cases} r_P^{(n)} [1 - \delta_{P_{loss}^{Rx}}(t), 0] + r_Q^{(n)}(t) \times \delta_{P_{loss}^{Rx}}(t), 0, & \text{if } n \in \Psi' \\ r_Q^{(n)}(t), & \text{if } n \in \Psi \end{cases} \quad (5)$$

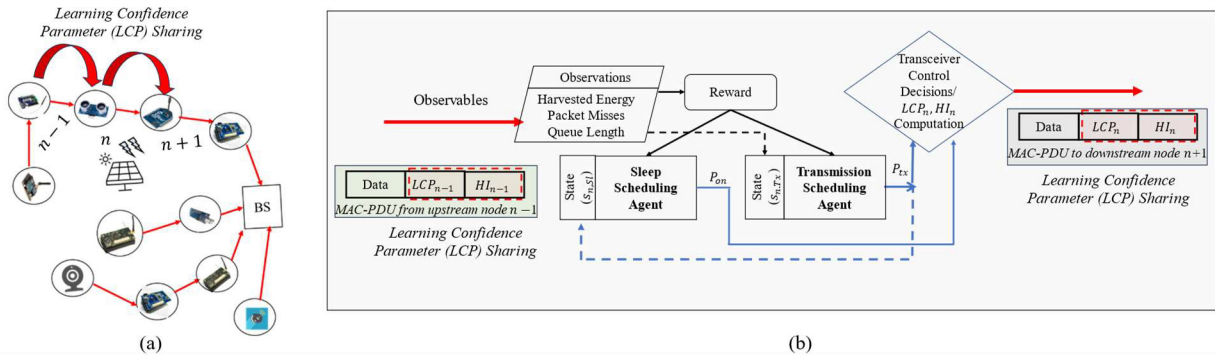


Fig. 1. Proposed Cooperative Multi-Agent Reinforcement Learning: LCP sharing for learning parameter update at the network level is shown in (a); while the system-level working from a node-perspective is shown in (b).

Here, $P_{loss}^{rx}(t)$ denote the number of missed packet reception at epoch t ; $\delta_{i,j}$ is the Kronecker delta function; Ψ is the set of source nodes in the network; and $r_P^{(n)}, r_Q^{(n)}$ are the reward function associated with reducing missed packet receptions and maintaining a sustainable MAC packet queue respectively, as defined in Eqns. (6) and (7) respectively.

$$r_P^{(n)}(t) = \frac{1}{P_{loss}^{rx}(t)} \quad (6)$$

$$r_Q^{(n)}(t) = \begin{cases} \frac{1}{\Delta Ql(t)} + \left(1 - \frac{1}{\Delta Ql(t)}\right)u(\Delta Ql(t) + \Gamma) + \frac{1 - \Delta Ql(t)}{2\Delta Ql(t)}\delta_{\Delta Ql(t), -\Gamma}, & \text{if } n \in \Psi' \\ sgn(\Delta Ql(t) + \Gamma) - \delta_{\Delta Ql(t), -\Gamma}, & \text{if } n \in \Psi \end{cases} \quad (7)$$

In Eqns. (6) and (7), u and $sgn(\cdot)$ represent unit step and signum functions respectively. $\Delta Ql(t) = Ql(t) - Ql(t-1)$ is the temporal gradient of the MAC packet queue length which needs to be negative for a stable queue. $\Gamma > 0$ is a hyperparameter to ensure that learning does not get stuck in a local optimal point.

The physical interpretation of the reward function can be explained as follows. As can be seen from Eqn. (5), there are two primary components of reward. First, the reward associated with successful packet reception $r_P^{(n)}$ is an inverse function of missed packet reception rate (Eqn. (6)). It means that for the high value of the missed packet reception rate, the agent receives a low value of reward. Similarly, in order to reduce the queueing delay, a negative temporal gradient of queue length is rewarded, which is captured by the reward component $r_Q^{(n)}(t)$. Note that, for a source node that is generating packets and has nothing to do with packet reception, the RL agent's only task is to reduce the packet delay and hence its reward only consists of the component $r_Q^{(n)}(t)$ (Eqns. (5), (6)).

Using the RL framework detailed above, the transmission scheduling agent learns to find a packet transmission schedule to reduce the missed packet receptions because of energy shortage and also to reduce the packet delay. However, to be noted that the transmission strategy learnt by this agent is contingent upon the transceiver on/off policy of the node. To exemplify, if the packet transmission probability of a node is P_{tx} and the node off probability is P_{off} , then the effective transmission rate of the node is given by $P_{tx} \times (1 - P_{off})$.

Thus, the node's sleep decisions indirectly affect the learning behavior of the transmission scheduling. In addition, efficient sleep decisions also have a role to play in reducing the packet missed receptions. This calls for the requirement of a sleep scheduling agent that can cooperate with the transmission scheduling agent to achieve the predefined goals.

B. Sleep Scheduling Agent

The role of this agent is to find a transceiver 'On/Off' schedule for the energy harvesting node it is part of. The goal of the agent is to find a schedule so that the limited energy budget of the node can be efficiently managed while maintaining reliable communication by reducing packet loss and packet delay. The action space ($\mathcal{A}_S$) of the sleep scheduling agent is given by the probability of keeping the radio transceiver *on* (P_{on}) in an RL decision epoch of h frames. This probability is discretized into $|\mathcal{A}_S|$ discrete values in the range $[0, 1]$, where $|\mathcal{A}_S|$ denotes the cardinality of the action space of the sleep scheduling agent. The actions are selected using ϵ -greedy policy to maximize the expected long-term expected reward using Q-learning approach discussed above.

The state space ($\mathcal{S}_S$) for this agent is defined by the probability of packet transmission (P_{tx}) by the node in a learning epoch. The probability P_{tx} and hence the state of the sleep scheduling agent is directly controlled by the learning policy of the transmission scheduling agent. The logic behind using P_{tx} as the state for this agent is that the packet transmission probability and the transceiver 'on' probability jointly determine the energy expenditure and throughput of the node. As a result, for a given transmission strategy decided by the transmission scheduler, this agent has to act accordingly to find a suitable transceiver *on* probability (P_{on}). Formally, the state perceived by the sleep scheduling agent for node n at a decision epoch t is given by Eqn. (8) ($\mathcal{S}_S \in I$ and $\mathcal{S}_S \geq 0$).

$$\begin{aligned} s_{n,Sl}(t) &= f(P_{Tx,n}(t)) \\ &= \mathcal{S}_s \text{ if } \mathcal{S}_s \leq 10 \times P_{Tx}(t) \leq \mathcal{S}_s + 1 \end{aligned} \quad (8)$$

Here, $P_{Tx,n}(t)$ is the probability of transmission by node n at epoch t . The actions taken by the sleep scheduling agent are evaluated using the same reward function given in Eqn (5), since both agents share the common objective of reducing missed packet receptions and delays. These two agents, thus,

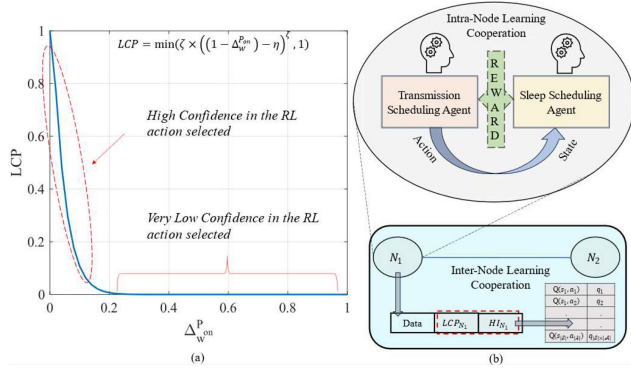


Fig. 2. (a) Computation of LCP, (b) Cooperation in Learning Transmit-Sleep Schedule.

cooperate with each other to jointly learn a policy that can achieve the above objective.

C. Learning Confidence Parameter Sharing

For achieving cooperative learning behavior, each node piggybacks a Learning Confidence Parameter (LCP) in the MAC layer PDU. This inter-node parameter sharing mechanism is demonstrated by the red arrows in Fig. 1. The figure also shows the incoming MAC-PDU (received from node ' $n - 1$ ') and outgoing MAC-PDU (destined for node ' $n + 1$ ') for the node n . This Learning Confidence Parameter represents the confidence level in the action selection policy for that node and is computed as given in Eqn. (9).

$$LCP(t) = \min\left(\zeta \times \left((1 - \Delta_w^{P_{on}}(t)) - \eta\right)^\zeta, 1\right) \quad (9)$$

Here, $\Delta_w^{P_{on}}(t) = P_{on}(t) - P_{on}(t - w)$ is the temporal gradient of transceiver on probability (P_{on}) over a window of w epochs. Learning confidence is low for a high value of this gradient $\Delta_w^{P_{on}}$ (more oscillatory actions) and vice-versa. A non-linear function is used to map $\Delta_w^{P_{on}}$ to Learning Confidence Parameter, so that for the range of values of $\Delta_w^{P_{on}}$ for which the confidence in learning is low increases. This can be seen in Fig. 2 (a), where LCP is plotted against $\Delta_w^{P_{on}}$ for $w = 1000$, $\zeta = 20$ and $\eta = 0.13$.

The significance of using LCP is that during the initial exploratory stage of learning, there may be bad action selection by a node. This leads to high missed packet receptions and thus affects the data rate of the flow it is part of. Hence, for the RL agents associated with the downstream nodes in the flow, it will lead to poor learning performance if their Q-values are updated when bad actions are chosen for the upstream nodes' agents. As for example, in the network shown in Fig. 1 (a), bad learning policies for node ' $n - 1$ ' would affect learning performance of node ' n ' and ' $n + 1$ ', if nodes ' n ' and ' $n + 1$ ' are not aware of confidence on the learning policies for node ' $n - 1$ '. Sharing the LCP helps avoid such scenarios. Because, while updating the Q-table, the temporal difference error is discounted by LCP for all nodes except the source. For the source node, the table update is done using the regular Bellman equation, since its learning is not dependent on the 'learning

confidence' of any other node in the network. The value update equation is given by:

$$Q_n(s_n, a_n) \leftarrow \begin{cases} Q_n(s_n, a_n) + LCP \times \alpha \left[\mathcal{R}^{(n)} + \gamma \max_{a'} Q(s'_n, a'_n) - Q(s_n, a_n) \right], & \text{if } n \in \Psi' \\ Q_n(s_n, a_n) + \alpha \left[\mathcal{R}^{(n)} + \gamma \max_{a'} Q_n(s'_n, a'_n) - Q_n(s_n, a_n) \right], & \text{if } n \in \Psi \end{cases} \quad (10)$$

Here s_n , s'_n , a_n , $\mathcal{R}^{(n)}$ denote the current state, next state, action, and reward respectively for node n . Note that the same update rule is used by both the Transmission Scheduling Agent and Sleep Scheduling agent. In addition to LCP , each node n also piggybacks the *Hop Index* (HI_n), defined as the number of hops the node is away from the source (Fig. 1). This information is used in determining the exponential decay rate in the ϵ -greedy exploratory behavior policy. Formally, the value of ϵ for a node n while following ϵ -greedy exploratory action selection (with learning delay of τ and exponential decay rate of ϵ_{dec}) is:

$$\epsilon_n(t) = \min\left(\epsilon_{max} e^{-\frac{t - HI_n \times \tau}{\epsilon_{dec}}}, 1\right) \quad (11)$$

D. Cooperation Among Learning Agents

As discussed in Section IV A–C, there are two RL agents deployed in each energy-harvesting sensor node that jointly learn the transmit-sleep schedules for that node. Moreover, each node shares a learning confidence parameter (LCP) with its one-hop neighbors in order to assist them with an informed-update of their own Q-tables. There are two levels of hierarchy in the Reinforcement Learning cooperation: intra-node cooperation and inter-node cooperation. This is demonstrated in Fig. 2 (b). The intra-node cooperation is between the two learning agents that jointly learn the transmit and sleep scheduling policies. The cooperation here is achieved by sharing the reward model and state information among the transmit and sleep scheduling learning agent towards the common objectives of minimizing packet loss and access delay. As has been discussed in Section IV A and B, the state of the sleep-scheduling RL agent is controlled by the policies learnt by the transmission scheduling agent. The second level of cooperation is across the nodes. This inter-node cooperation is shown in Fig. 2 (b), where nodes N_1 and N_2 cooperate by sharing learning control information, which is then used to update their respective Q-values. This ensures that the learning parameters are not updated for observables that are not reliable. The Q-table update is performed in each node in a decentralized manner, thus making the framework scalable with network size and degree.

V. EXPERIMENTATION

Experiments were performed to analyze the performance of the proposed scheduling protocol using a MAC layer simulator with embedded learning components. The baseline experimental parameters are tabulated in Table II. The performance of the scheduling policy learnt using the proposed RL-framework is compared with the following scheduling approaches.

TABLE II
BASELINE EXPERIMENTAL PARAMETERS

Parameter	Value
P_C, w_s, C	0.2, 33.33 m/s, 50 m
α, γ, h	0.99, 0.1, 200
B, P_{charge}, Ql_{max}	150, 0.9, 1000
$ \mathcal{A}_T = \mathcal{A}_S $	10
τ_1, τ_2	100, 1
ν, δ, α_T	0.90, 0.001, 2.4
E_{tx}, E_{ct}	0.353 mW, 50 μ W

Naïve Scheduling Policy: This is the baseline policy where a node is ‘On’ in a frame with probability P_{on} and it transmits with probability P_{tx} . These probabilities are manually preset to fixed values and experiments are performed with different combinations of these probabilities to understand the variation of the performance variables (packet loss rate and packet delay) as response to these sleep-transmit decisions.

Battery State-based policy: This is an existing scheduling approach proposed in [2]. Here the node sleep decisions are based on the amount of available battery power in the sensor node. The node goes from active to sleep mode if the normalized battery capacity is below a preset threshold b_{low} and wakes up again when the battery state is above the threshold b_{high} . Mathematically, the transition probabilities from active to sleep states and vice-versa are formulated by Eqn. (12):

$$P_{a \rightarrow s} = \begin{cases} 1, & \text{if } \frac{b}{B} \leq b_{low} \\ 0, & \text{if } \frac{b}{B} > b_{low} \end{cases} \quad P_{s \rightarrow a} = \begin{cases} 1, & \text{if } \frac{b}{B} \geq b_{high} \\ 0, & \text{if } \frac{b}{B} < b_{high} \end{cases} \quad (12)$$

Queue length-based policy: This scheduling strategy decides sleep schedules based on the MAC queue length. The node sleeps with high probability if the queue length is higher than a predefined threshold Ql_{th} and vice versa.

$$P_{a \rightarrow s} = \begin{cases} \delta_1, & \text{if } Ql \leq Ql_{th} \\ \delta_2, & \text{if } Ql > Ql_{th} \end{cases} \quad \text{and} \quad P_{s \rightarrow a} = \begin{cases} \delta_3, & \text{if } Ql \leq Ql_{th} \\ \delta_4, & \text{if } Ql > Ql_{th} \end{cases} \quad (13)$$

Solar radiation-based policy: This scheduling logic makes the node go to sleep with probability ϕ_s and to wake up with probability 1 and ϕ_w , when the solar radiation state (r) is low and high respectively. Formally, this can be expressed as

$$P_{a \rightarrow s} = \begin{cases} \phi_s, & \text{if } r = 0 \\ 0, & \text{if } r > 0 \end{cases} \quad \text{and} \quad P_{s \rightarrow a} = \begin{cases} \phi_w, & \text{if } r = 0 \\ 1, & \text{if } r > 0 \end{cases} \quad (14)$$

Hybrid policy: The hybrid scheduling policy takes both queue length and battery power into consideration for making scheduling decisions. This hybrid policy is given by Eqn. (15):

$$P_{a \rightarrow s} = \begin{cases} 1, & \text{if } \frac{b}{B} \leq b_{low} \text{ or } Ql = 0 \\ 0, & \text{if } \frac{b}{B} > b_{low} \end{cases} \quad (15)$$

$$P_{s \rightarrow a} = \begin{cases} 1, & \text{if } \frac{b}{B} \geq b_{high} \\ 0, & \text{if } \frac{b}{B} < b_{high} \end{cases}$$

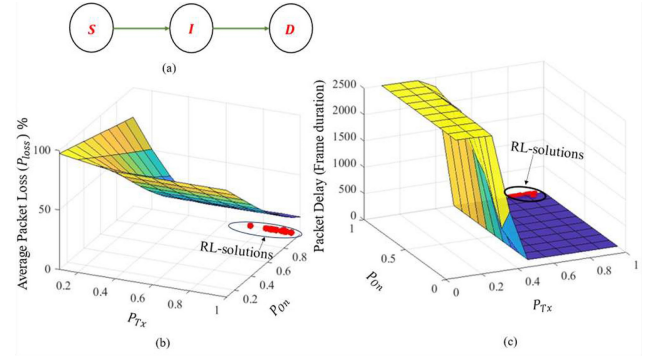


Fig. 3. Average packet loss and delay with naïve policy and proposed RL-based policy (node I is the intermediate node).

Note that the existing scheduling strategies mentioned above: *Battery State-based*, *Queue length-based*, *Solar radiation-based* and *hybrid policies* are proposed in [2] and default policy hyperparameters are chosen for comparison.

VI. RESULTS AND ANALYSIS

With the aim of understanding the working of the proposed framework, we first experiment with a simple linear topology and then with more complex topologies. For the linear topology case, a source node transmits packets at a rate of $\lambda = 0.4$ packet per frame (PPF). First, to get an insight into the baseline performance, we experiment with the naïve scheduling policy, where the node is kept ‘On’ in a frame with probability P_{on} and it transmits with probability P_{tx} . The surface plot in Fig. 3 (b) shows the variation of packet loss rate P_{loss} for different values of P_{on} and P_{tx} . It is observed that with an increase in ‘node on’ probability P_{on} , there is a decrease in P_{loss} . This is because, with an increase in the value of P_{on} , it is more likely for the node to remain On in a frame and hence there is less probability of missed packet reception. Also for low values of P_{tx} , the gradient of decrease in P_{loss} with P_{on} decreases. This is because, for low probability of packet transmission, queues build up and get full, resulting in packet drops. As a result, for high probability of the node being On, even if the missed packet reception is low, still there is high P_{loss} because of packet drops resulting from full MAC packet queues. Fig. 3 (c) demonstrates the variation of queuing delay with the scheduling probabilities P_{tx} and P_{on} . As seen from the figure, the queuing delay decreases with an increase in P_{tx} . This is because with an increase in P_{tx} , missed packet receptions increase and there are less packets received. Also, the queue service rate increases with an increase in P_{tx} . This causes the queue length, and hence queuing delay, to decrease with P_{tx} . Note that there is a range of values of P_{tx} for which the gradient in decrease in packet delay is very high. This range represents the scenario when the queue service rate μ gets close to the effective flow data rate λ_{eff} . In other words, when $\mu < \lambda_{eff}$, the queue is stable and packet delay is low; and when $\mu > \lambda_{eff}$, it leads to unstable queue and packet delay is very high. However, the effects of P_{on} on queuing delay is not significant. This is because, the expected queue length ($\mathbb{E}[N_Q]$) as defined by Eqn. (16) does not get affected

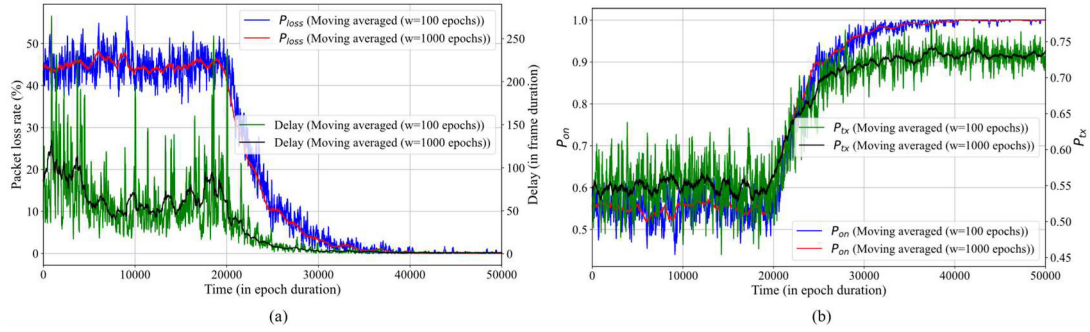


Fig. 4. RL convergence behavior with respect to (a) Packet loss rate and delay and (b) sleep and transmit probabilities.

by P_{on} , since the flow rate and service rate both are affected by P_{on} by the same amount.

$$\mathbb{E}[N_Q] \propto \frac{\rho^2}{1-\rho}, \text{ where } \rho = \frac{p_{on} \times \lambda}{p_{on} \times \mu} = \frac{\lambda}{\mu}. \quad (16)$$

The performance achieved by the RL-based scheduling approach is shown by the red points on the surface plot in Fig. 3. It can be observed that the proposed RL-based mechanism finds sleep and transmission schedules that give lower packet loss rate and packet delay than the static naïve scheduling policy. Note that the schedules learnt by the cooperative RL approach have lower value of P_{loss} for the same P_{tx} , P_{on} values used in the naïve scheduling approach. This is because the RL approach allows the nodes to learn dynamic sleep-transmit scheduling. In other words, the learnt scheduling probabilities oscillate epoch by epoch. To exemplify, a transmit probability of 0.6 can indicate refraining from transmission deterministically for four consecutive epochs (800 frames), thus recharging its battery, and then transmitting for next six epochs (1200 frames), thus, utilizing its recharged battery. It was observed that such dynamic policies learnt by the RL agents helped the node to obtain a packet loss (P_{loss}) rate less than the static naïve policies for the same $\langle P_{tx}, P_{on} \rangle$ tuple.

The learning convergence behavior of the proposed architecture in terms of packet loss rate and packet delay is shown in Fig. 4 (a) and (b). It is observed that initially, at the start of the learning, P_{loss} and packet delay is high; but over time, the nodes learn to find transmission and sleep scheduling policies, so that both packet loss rate and packet delay go down. Another observation is that on an average, the learning convergence time remains in the vicinity of 10^4 frame durations. For a typical MAC frame duration of 3-4 ms [22], the RL convergence can happen within 30 - 40 sec. This makes the approach highly practical in that it can cope with network condition changes with a time constants that are order of 10s to 100s of minutes. For many static (i.e., non-mobile) sensor networks, the time constants for network topology and traffic condition changes can be even larger, often up to hours to days.

Next, we compare its performance with the existing sleep-scheduling protocols: *Battery State-based*, *Queue length-based*, *Solar radiation-based* and *hybrid policies*. The comparison is made for topologies with multiple hops between

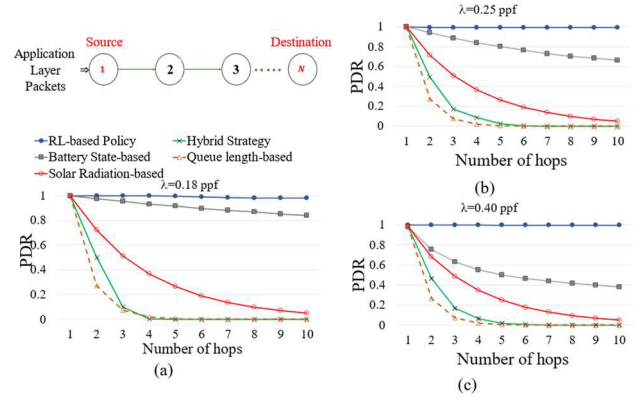


Fig. 5. Performance comparison of proposed RL-based protocol with existing approaches for a linear topology with multiple hops.

the source and destination is shown in Fig. 5. It is observed that with an increase in the number of hops between source and destination nodes, the packet delivery ratio (PDR) falls more drastically for the other scheduling logics as compared to that of the proposed RL-based scheduling approach. The reason behind this behavior is that these baseline policies are designed for energy saving, specifically from the perspective of an individual node, without considering the energy management of the network as a whole. On the other hand, the proposed scheduling mechanism allows the nodes to cooperatively learn the transmit and sleep scheduling strategy by sharing the online Learning Confidence Parameter (LCP) within the localized neighborhood. This piggybacked information is used by the learning agents to avoid bad updates of Q-table (as explained in Section IV) and helps in maintaining performance for networks where the source and destination nodes are separated by many numbers of hops. As seen from the figure, with the increase in flow data rate, the benefit of the proposed framework in terms of PDR becomes more significant. This is because, for high data rate, there is a high packet loss rate due to energy shortage when the existing scheduling approaches are used. But, using the proposed learning-based scheme, the transmit and sleep scheduling agents learn cooperatively to find a joint transmit-sleep schedule to reduce the packet losses and hence to achieve a high packet delivery ratio.

To test the generalizability of the proposed framework, we experimented with heterogeneous networks consisting of

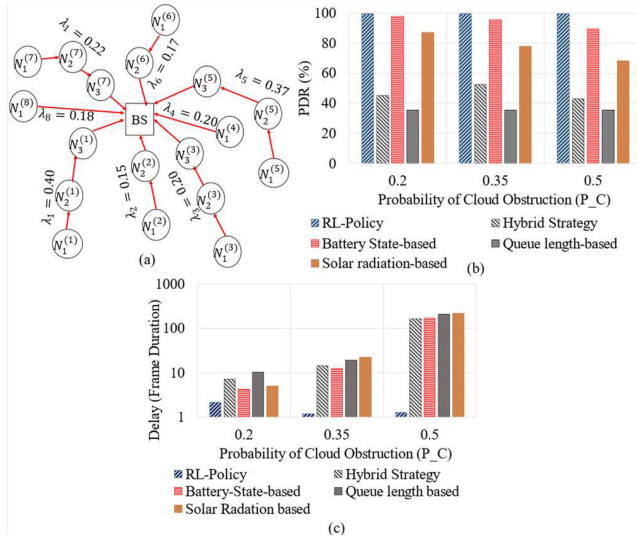


Fig. 6. Performance comparison of proposed RL-based protocol with existing approaches for different solar radiation conditions in a heterogeneous topology with 8 flows.

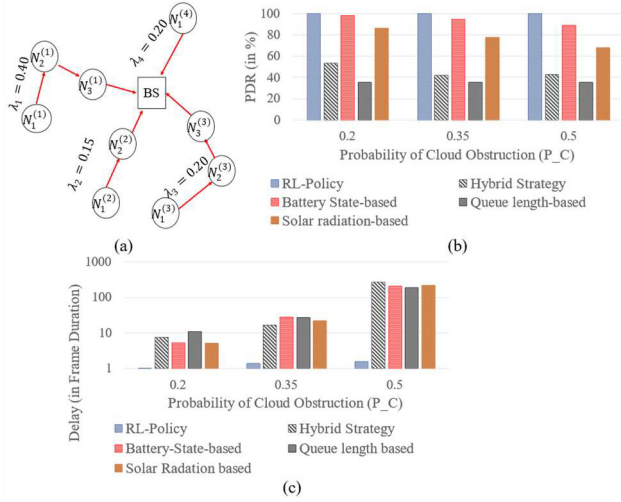


Fig. 7. Performance comparison of proposed RL-based protocol with existing approaches for different solar radiation conditions in a heterogeneous topology with 4 flows.

multiple flows with different flow lengths and different Poisson data-rates in each flow (Figs. 6 (a) and 7 (a)). Two networks (with 8 and 4 flows) are considered with multi-point to point traffic, where the flow rates are shown in the figures. The plots in Figs. 6 and 7 compare the performance of the proposed RL-based scheduling policy with the other baseline approaches mentioned above. It is observed that for different values of cloud obstruction probability (P_C), the packet delivery ratio is high and end-to-end packet delay is less for the proposed RL-based scheduling logic. With an increase in cloud obstruction probability, packet drops due to energy shortage increases for the *Battery State-based*, *Queue length-based*, *Solar radiation-based*, and *hybrid sleep scheduling policies*. However, the learning-driven transmit-sleep scheduling strategy makes the nodes learn to manage the energy budget so that the packet delivery ratio remains high, and packet delay remains low. This makes it suitable for energy-constrained wireless sensor

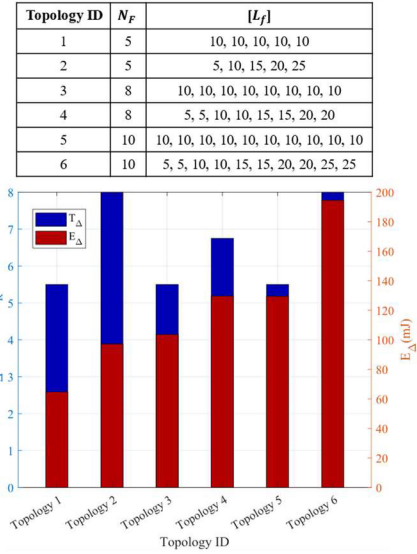


Fig. 8. Performance comparison of proposed RL-based protocol with DRL-driven scheduler proposed in [10].

and IoT networks to efficiently utilize resources to maintain sustainable communication.

Comparison with state-of-the-art learning-based scheduling: We compare the performance of the proposed RL-based scheduling algorithm with the existing DRL-driven sleep-scheduling framework developed in [10]. The main advantages of the proposed scheme as compared to that mentioned in [10] are its generalizability to multi-hops networks and the fully decentralized implementation. We compare the performance of these two RL-driven approaches based on two metrics: (a) additional energy expenditure (E_{Δ}), and (b) average additional latency incurred (T_{Δ}) while uploading the learning observables to the central controller and downloading the learnt policies by the nodes. These two metrics can be defined by the following equations.

$$E_{\Delta} = \sum_{f=1}^{N_F} (L_f \times P_T + (L_f - 1) \times P_R) \quad (17)$$

$$T_{\Delta} = \frac{1}{N_F} \sum_{f=1}^{N_F} \frac{(L_f + 1)}{2} \tau_R \quad (18)$$

Here N_F , L_f represent the number of flows and number of nodes in flow f , in a multi-hop star-topology network; τ_R is the round-trip propagation delay (which is assumed to be equal for each hop). In this model, we assume that the processing time is much smaller than the round-trip delay. Fig. 8 shows the additional energy consumption and average latency incurred by the DRL-based scheduling architecture developed in [10], as compared to the transmit-sleep scheduler proposed here. For the transmit and receive power computation in this figure, we use the default parameters as mentioned earlier. The comparison is done for six different topologies of 5, 8 and 10 flows with different number of hops. It shows that even for a single parameter update, the additional energy spent is close to 200 mJ and the average latency incurred is more than $7 \tau_R$.

VII. SUMMARY AND CONCLUSION

In this paper, a cooperative Reinforcement Learning architecture is developed for energy management in energy-harvesting wireless sensor networks. This decentralized framework allows the nodes to learn a joint transmit-sleep scheduling policy in order to efficiently utilize the lean energy budget of the sensor nodes. And it is done while maintaining reliable communication in terms of low packet loss rate and packet delay. Using a cooperative two-agents RL model, the nodes learn a situation-specific transmit-sleep schedule. An online learning confidence parameter sharing module allows the nodes to learn scheduling policies by ignoring unreliable RL observations. This makes the system scalable for long flows with many hops between the source and destination. With simulation studies, the developed RL-based scheduling policy is shown to outperform existing MAC sleep schedulers, especially for networks with multiple hops between source and destination. Simulation results demonstrate that the proposed learning-driven scheduling approach achieves a packet delivery ratio that is $\approx 60\%$ higher than the existing mechanisms for a network with 10-hops between source and destination. Moreover, for adverse energy harvesting conditions, the developed framework is shown to achieve $\approx 10\%$ increase in packet delivery ratio and two-orders of magnitude lower end-to-end delay than the existing scheduling mechanisms.

REFERENCES

- [1] W. Ye, J. Heidemann, and D. Estrin, "An energy-efficient MAC protocol for wireless sensor networks," in *Proc. 21st Annu. Joint Conf. IEEE Comput. Commun. Soc.*, 2002, pp. 1567–1576.
- [2] D. Niyato, E. Hossain, and A. Fallahi, "Sleep and wakeup strategies in solar-powered wireless sensor/mesh networks: Performance analysis and optimization," *IEEE Trans. Mobile Comput.*, vol. 6, no. 2, pp. 221–236, Feb. 2007.
- [3] G. Chen, Y. Zhan, Y. Chen, L. Xiao, Y. Wang, and N. An, "Reinforcement learning based power control for in-body sensors in WBANs against jamming," *IEEE Access*, vol. 6, pp. 37403–37412, 2018.
- [4] G. Chen, Y. Zhan, G. Sheng, L. Xiao, and Y. Wang, "Reinforcement learning-based sensor access control for WBANs," *IEEE Access*, vol. 7, pp. 8483–8494, 2019.
- [5] X. Wang, H. Chen, and S. Li, "A reinforcement learning-based sleep scheduling algorithm for compressive data gathering in wireless sensor networks," *EURASIP J. Wireless Commun. Netw.*, vol. 2023, no. 1, p. 28, 2023.
- [6] R. Mohammadi and Z. Shirmohammadi, "RLS2: An energy efficient reinforcement learning-based sleep scheduling for energy harvesting WBANs," *Comput. Netw.*, vol. 229, Jun. 2023, Art. no. 109781.
- [7] A. K. Sangaiah, A. Javadpour, F. Ja'fari, H. Zavieh, and S. M. Khaniabadi, "SALA-IoT: Self-reduced Internet of Things with learning automaton sleep scheduling algorithm," *IEEE Sensors J.*, vol. 23, no. 18, pp. 20737–20744, Sep. 2023.
- [8] X. Cao, W. Xu, X. Liu, J. Peng, and T. Liu, "A deep reinforcement learning-based on-demand charging algorithm for wireless rechargeable sensor networks," *Ad Hoc Netw.*, vol. 110, Jan. 2021, Art. no. 102278.
- [9] Y.-H. Xu, J.-W. Xie, Y.-G. Zhang, M. Hua, and W. Zhou, "Reinforcement learning (RL)-based energy efficient resource allocation for energy harvesting-powered wireless body area network," *Sensors*, vol. 20, no. 1, p. 44, 2019.
- [10] L. Wang, G. Zhang, J. Li, and G. Lin, "Joint optimization of power control and time slot allocation for wireless body area networks via deep reinforcement learning," *Wireless Netw.*, vol. 26, pp. 4507–4516, May 2020.
- [11] S. N. Das, S. Misra, B. E. Wolfinger, and M. S. Obaidat, "Temporal-correlation-aware dynamic self-management of wireless sensor networks," *IEEE Trans. Ind. Informat.*, vol. 12, no. 6, pp. 2127–2138, Dec. 2016.
- [12] Y.-H. Xu, G. Yu, and Y.-T. Yong, "Deep reinforcement learning-based resource scheduling strategy for reliability-oriented wireless body area networks," *IEEE Sens. Lett.*, vol. 5, no. 1, pp. 1–4, Jan. 2021.
- [13] Z. Liu, B. Liu, and C. W. Chen, "Joint power-rate-slot resource allocation in energy harvesting-powered wireless body area networks," *IEEE Trans. Veh. Technol.*, vol. 67, no. 12, pp. 12152–12164, Dec. 2018.
- [14] S. Radhika and P. Rangarajan, "Fuzzy based sleep scheduling algorithm with machine learning techniques to enhance energy efficiency in wireless sensor networks," *Wireless Pers. Commun.*, vol. 118, pp. 3025–3044, Feb. 2021.
- [15] T. Rana, A. Sett, K. Biswas, and T. Saha, "Fuzzy-based clustering toward improving the lifespan of wireless sensor networks," in *Proc. Int. Conf. Adv. Comput. Appl.*, 2022, pp. 369–380.
- [16] Z. Guo and H. Chen, "A reinforcement learning-based sleep scheduling algorithm for cooperative computing in event-driven wireless sensor networks," *Ad Hoc Netw.*, vol. 130, May 2022, Art. no. 102837.
- [17] N. Zhu, X. Xu, S. Han, and S. Lv, "Sleep-scheduling and joint computation-communication resource allocation in MEC networks for 5G IIoT," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2021, pp. 1–7.
- [18] W. B. Powell, *Reinforcement Learning and Stochastic Optimization: A Unified Framework for Sequential Decisions*. Hoboken, NJ, USA: Wiley, 2022.
- [19] T. Li et al., "Applications of multi-agent reinforcement learning in future Internet: A comprehensive survey," *IEEE Commun. Surveys Tuts.*, vol. 24, no. 2, pp. 1240–1279, 2nd Quart., 2022.
- [20] R. S. Sutton et al., *Reinforcement Learning: An Introduction*. Cambridge, MA, USA: MIT Press, 2018.
- [21] L. P. Kaelbling, M. L. Littman, and A. W. Moore, "Reinforcement learning: A survey," *J. Artif. Intell. Res.*, vol. 4, pp. 237–285, May 1996.
- [22] X. Zhang, X. Jiang, and M. Zhang, "A black-burst based time slot acquisition scheme for the hybrid TDMA/CSMA multichannel MAC in VANETs," *IEEE Wireless Commun. Lett.*, vol. 8, no. 1, pp. 137–140, Feb. 2019.
- [23] H. Dutta and S. Biswas, "Towards multi-agent reinforcement learning for wireless network protocol synthesis," in *Proc. Int. Conf. Commun. Syst. Netw. (COMSNETS)*, 2021, pp. 614–622.
- [24] M. Courbariaux, I. Hubara, D. Soudry, R. El-Yaniv, and Y. Bengio, "Binarized neural networks: Training neural networks with weights and activations constrained to+ 1 or -1," 2016, *arXiv:1602.02830*.
- [25] D. Nitnaware and A. Verma, "Performance evaluation of energy consumption of reactive protocols under self-similar traffic," *Int. J. Comput. Sci. Commun.*, vol. 1, no. 1, pp. 67–71, 2010.
- [26] Z. Yao, S. Xia, Y. Li, and G. Wu, "Cooperative task offloading and service caching for digital twin edge networks: A graph attention multi-agent reinforcement learning approach," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3401–3413, Nov. 2023.
- [27] Y. Wang, Y. Wu, Y. Tang, Q. Li, and H. He, "Cooperative energy management and eco-driving of plug-in hybrid electric vehicles via multi-agent reinforcement learning," *Appl. Energy*, vol. 332, Feb. 2023, Art. no. 120563.
- [28] H. Wang, S. Wang, Y. Yang, and D. Zhang, "GCRL: Efficient delivery area assignment for last-mile logistics with group-based cooperative reinforcement learning," in *Proc. Int. Conf. Data Eng.*, 2023, pp. 3522–3534.
- [29] G. Shen, L. Lei, X. Zhang, Z. Li, S. Cai, and L. Zhang, "Multi-UAV cooperative search based on reinforcement learning with a digital twin driven training framework," *IEEE Trans. Veh. Technol.*, vol. 72, no. 7, pp. 8354–8368, Jul. 2023.
- [30] A. Oroojlooy and D. Hajinezhad, "A review of cooperative multi-agent deep reinforcement learning," *Appl. Intell.*, vol. 53, pp. 13677–13722, Jun. 2023.



Hrishikesh Dutta (Member, IEEE) received the Ph.D. degree from Michigan State University in 2024. He is working as a Postdoctoral Researcher with Institut Polytechnique de Paris. His research interests include Internet-of-Things, computer networks, and machine learning.



Amit Kumar Bhuyan (Graduate Student Member, IEEE) is currently pursuing the Ph.D. degree with the Electronics and Communication Engineering Department, Michigan State University. His research interests include vehicular communication, distributed systems, federated learning, reinforcement learning, and signal processing.



Subir Biswas (Senior Member, IEEE) received the Ph.D. degree from the University of Cambridge and held various research and management positions in NEC Research Institute, Princeton, AT&T Laboratories, Cambridge, and Tellium Optical Systems, NJ, USA. He is a Professor and the Director of the Networked Embedded and Wireless Systems Laboratory, Michigan State University. His current research includes data dissemination in vehicular networks, wearable systems for health applications, and learning in embedded and resource-constrained systems.